🚕 CABB - Crunch Alma Bibs in Bulk

A Flet-based single-page UI application designed to perform various Alma-Digital bibliographic record editing functions using *mostly* the Alma API.

🚕 **CABB** (Crunch Alma Bibs in Bulk) provides a user-friendly interface for cleaning and maintaining Alma bibliographic records, supporting both single-record operations and batch processing via Alma Sets or CSV MMS ID lists.

Features

The application supports both **single record** and **batch processing** (via Alma Sets):

Processing Modes

1. **Single Record Mode** - Enter an MMS ID to process one bibliographic record at a time
2. **Batch Processing Mode** - Enter a Set ID or a CSV file path to load members (all or limited) and process them in bulk

Editing Functions

Functions are split into **Active** (shown first in the dropdown) and **Inactive** (less frequently used) categories.

Active Functions

- **1: Fetch and Display Single XML** - Preview the full XML structure of a bibliographic record
- **3: Export Set to DCAP01 CSV** - Export Dublin Core metadata from a set to CSV format
- **5: Get IIIF Manifest and Canvas** - Retrieve IIIF manifest and canvas information for digital objects
- **8: Export dc:identifier CSV** - Export all identifier fields to CSV for analysis
- **9: Validate Handle URLs and Export Results** - Test Handle URLs and report HTTP status codes
- **10: Export for Review with Clickable Handles** - Generate review spreadsheet with clickable Handle links and empty assessment columns
- **11: Prepare TIFF/JPG Representations** - Create TIFF and JPG representations using the full Alma API approach (Corinna/Harvard spec)
- **12: Analyze Sound Records by Decade** - Examine records where `dc:type` is "sound", extract year from `dc:date`/`dcterms:created`, and export a CSV grouped by decade
- **14a: Prepare Thumbnails (Part 1 of 2)** - Create `.clientThumb` representations and prepare thumbnail image files for Alma upload
- **14b: Upload Thumbnails (Part 2 of 2)** - Upload prepared thumbnail files to Alma representations via Selenium browser automation
- **15: Analyze dc:identifier Match with MMS ID** - Categorize records based on whether they already have a `dc:identifier` that exactly matches the MMS ID; exports up to three CSVs (matching / non-matching / failed)
- **16: Add MMS ID as dc:identifier** - Add the bare MMS ID as a `dc:identifier` field if not already present; handles duplicate identifier cleanup

- **17: Restore Metadata from Previous Version** - Automate the Alma MDE "View Versions → Restore" workflow via Selenium/Chrome to restore bibliographic metadata from a prior version
- **18: Identify Single TIFF Objects** - Scan a set and report digital objects that have only one TIFF representation (likely candidates for JPG derivative creation)

Inactive Functions

- **2: Clear dc:relation Collections Fields** - Removes all `dc:relation` fields whose value begins with `alma:01GCL_INST/bibs/collections/`
- **4: Filter CSV for Records 95+ Years Old** - Filter an exported CSV to retain only records with a publication date before 1930
- **6: Replace old dc:rights with Public Domain link** - Update legacy copyright statements to a standardized Public Domain URI
- **7: Add Grinnell: dc:identifier Field As Needed** - Add an institution-specific identifier derived from existing `dg_*` identifier values
- **11b: Upload JPG Files** *(DISABLED — Selenium approach abandoned)* - Former Selenium-based JPG upload workflow; replaced by the API-based Function 11
- **13: (Intentionally Left Blank)** - Reserved for triskaidekaphobia compliance

Set Processing

- Load any Alma set by Set ID
- Preview set members (first 20 displayed)
- Apply editing functions to all members in the set
- Real-time progress tracking during batch operations
- Success/failure summary after batch completion

Technology Stack

- **Flet 0.25.2** - Modern Python framework for building user interfaces (pinned — both `flet` and `flet-desktop` must be 0.25.2; newer versions may break `WebRenderer.HTML` or `Dropdown` initialization)
- **requests** - HTTP library for direct Alma API calls
- **python-dotenv** - Environment variable management
- **Python 3.x** - Programming language

Project Structure

```
CABB/
├── app.py              # Main Flet application
├── requirements.txt    # Python dependencies
├── run.sh             # Quick launch script
├── .env.example       # Environment variables template
├── .gitignore         # Git ignore rules
└── README.md          # This file
```

Installation

Prerequisites

- Python 3.8 or higher
- pip (Python package installer)
- An Alma API key with appropriate permissions

Setup Instructions

1. **Clone the repository**

```
git clone https://github.com/Digital-Grinnell/CABB.git
cd CABB
```

2. **Configure environment variables**

   Copy the example environment file and edit it with your Alma API credentials:

```
cp .env.example .env
```

   Edit `.env` and add your Alma API credentials:

```
ALMA_API_KEY=your_actual_api_key_here
ALMA_API_REGION=America
```

   Valid ALMA_API_REGION values:

   - `America` (default - North America)
   - `Europe`
   - `Asia Pacific`
   - `Canada`
   - `China`

   Required API Key Permissions:

   Your Alma API key must have the following permissions enabled:

   - **Bibs** - Read/Write (for reading and updating bibliographic records)
   - **Configuration** - Read-only (for accessing Sets and retrieving set members)

   To configure your API key in Alma:

   1. Go to Alma > Admin > General > API Key Management
   2. Create or edit your API key
   3. Ensure "Bibs" has Read/Write permissions
   4. Ensure "Configuration" has Read permissions
   5. Save the API key

3. **Run the application**

   Use the provided quick launch script:

   ```
   ./run.sh
   ```

   The script will:

   - Create a virtual environment (`.venv`) if it doesn't exist
   - Install all required dependencies
   - Launch the Flet application

Manual Installation (Alternative)

If you prefer to set up manually:

```
# Create virtual environment
python3 -m venv .venv

# Activate virtual environment
source .venv/bin/activate  # On Windows: .venv\Scripts\activate

# Install dependencies
pip install -r requirements.txt

# Run the application
python app.py
```

Optional: Selenium Setup (Functions 14b and 17)

Two functions use Selenium browser automation to interact with the Alma web interface for operations that are not exposed by the Alma REST API.

Function 14b — Upload Thumbnails (Firefox / GeckoDriver)

Function 14b uses Selenium + Firefox to upload thumbnail files via the Alma web interface (required because the Alma API file upload endpoint has limitations).

1. **Install GeckoDriver** (Firefox driver for Selenium):

   ```
   # macOS (via Homebrew)
   brew install geckodriver

   # Verify installation
   geckodriver --version
   ```

2. **How it works**:
    - Function 14b launches a NEW Firefox window automatically
    - Browser navigates to Alma SSO login page
    - You get 60 seconds to log into Alma via Grinnell SSO + DUO
    - If you need more time, automation pauses 30 additional seconds
    - Automation begins after login
    - Do not interact with Firefox during uploads

Note: Selenium cannot attach to an existing Firefox session—it must launch its own window. This is a Selenium/GeckoDriver design limitation.

See FUNCTION_14b_UPLOAD_THUMBNAILS.md for detailed instructions.

Function 17 — Restore Metadata (Chrome / ChromeDriver)

Function 17 uses Selenium + Chrome to automate the Alma Metadata Editor (MDE) "View Versions → Restore" workflow (the Alma REST API does not expose a `/bibs/{mms_id}/versions` endpoint).

1. **Install ChromeDriver** matching your installed Chrome version:

```
# macOS (via Homebrew)
brew install chromedriver

# Verify installation
chromedriver --version
```

2. **How it works**:
    - Function 17 launches a NEW Chrome window automatically
    - Navigates to the Alma SSO login page
    - You have 60 seconds to log in via Grinnell SSO + DUO
    - For each MMS ID, the automation opens the record in MDE, navigates to View Related Data → View Versions, and clicks Restore on the most recent prior version
    - A CSV audit report is written with per-record outcomes

See FUNCTION_17_RESTORE_METADATA.md for detailed instructions.

Usage

1. **Launch the application** using `./run.sh`

2. **Connect to Alma API** by clicking the "Connect to Alma API" button

Single Record Processing

3. **Enter an MMS ID** in the "Single Record" input field

4. **Select an editing function** from the available buttons:

- "Fetch and Display XML" to preview the record
- "Clear dc:relation Collections Fields" to remove collection references
- Additional functions as implemented

Batch Processing with Sets

3. **Enter a Set ID** in the "Batch Processing (Set)" input field
   - Example: `7071087320004641`
4. **Optional: Set a limit** in the "Limit" field to process a subset of records:
   - **0** (default) - Process all records in the set
   - **Positive number** (e.g., `200`) - Process only the **first** N records
   - **Negative number** (e.g., `-200`) - Process only the **last** N records

   Use case for negative limits:

   - If a long-running process stops at 98%, you can resume by using a negative limit
   - Example: Set had 12,000 records, stopped at 98% → Use `-240` to process the last 2%
   - This applies to **all functions** - the limit is applied when loading the set
5. **Click "Load Set Members"** to fetch all bibliographic records in the set
   - The app will display the set name and member count
   - First 20 members are shown for preview
   - If a limit is set, the display will indicate which records were loaded (first/last N)
6. **Select an editing function** to apply to all records in the set
   - Progress will be shown as "Processing X/Y: [MMS_ID]"
   - A summary of successes/failures is displayed upon completion
7. **Click "Clear Set"** when done to reset for single record or a different set

Viewing Logs

- The **Log Output** window shows real-time operation details
- Full logs are saved to timestamped files: `cabb_YYYYMMDD_HHMMSS.log`
- Click the copy icon next to "Status" to copy status messages to clipboard

2. **Connect to Alma API**
   - Click the "Connect to Alma API" button
   - The status will show if the connection is successful
3. **Enter a record MMS ID**
   - Input the bibliographic record's MMS ID in the text field
4. **Execute editing functions**
   - Click any of the five function buttons to perform operations on the specified record

    - The status area will display the result of each operation

Development

Project Development Log

Initial Setup (2024)

- Created project structure with virtual environment support
- Implemented Flet-based single-page UI
- Integrated almapipy library for Alma API connectivity
- Created five function placeholders with corresponding UI buttons

Function 1: Clear dc:relation Collections

- Implemented placeholder for clearing dc:relation fields
- Pattern matching: `alma:01GCL_INST/bibs/collections/*`
- Uses XML parsing for record manipulation
- Note: Full implementation requires actual XML manipulation logic

Architecture Decisions

- **Single-page design**: All functionality on one page for simplicity
- **Class-based architecture**: `AlmaBibEditor` class encapsulates all business logic
- **Environment variables**: API keys stored securely in `.env` file
- **Virtual environment**: Isolated Python environment for dependency management

Code Structure

`app.py`

The main application file containing:

- `AlmaBibEditor` class: Handles Alma API interactions
- `main(page)` function: Builds the Flet UI
- Event handlers for each button
- Status update mechanisms

Key Functions

`initialize_alma_connection()`

- Establishes connection to Alma API using almapipy
- Validates API key from environment variables
- Returns success status and message

`clear_dc_relation_collections(mms_id)`

- Main implementation for Function 1
- Fetches bibliographic record by MMS ID

- Parses XML to find and remove matching dc:relation fields
- Updates the record via Alma API

Placeholder Functions 2-5

- Reserved for future functionality
- Follow same pattern: accept MMS ID, return status tuple
- Ready for implementation of additional editing features

Adding New Functions

To implement additional editing functions:

1. Add a method to the `AlmaBibEditor` class
2. Create an event handler in the `main()` function
3. Add a button to the UI
4. Update the README with the new functionality

Example:

```
def new_editing_function(self, mms_id: str) -> tuple[bool, str]:
    """Description of new function"""
    # Implementation here
    return True, "Success message"
```

Dependencies

See `requirements.txt` for complete list:

- `flet==0.25.2` - UI framework (**pinned** — must match `flet-desktop==0.25.2`)
- `flet-desktop==0.25.2` - Desktop renderer (**pinned** — must match `flet` version)
- `python-dotenv>=1.0.0` - Environment variable management

API Documentation

- Alma API Documentation
- almapipy GitHub Repository
- Flet Documentation

Security Notes

- Never commit your `.env` file or API keys to version control
- The `.env` file is included in `.gitignore` to prevent accidental commits
- Use API keys with minimal required permissions
- Regularly rotate your API keys

Alma API Namespace Handling

This application uses a direct XML approach for updating Alma bibliographic records. Understanding how to properly handle XML namespaces is critical for successful API interactions.

The Challenge

Alma's API has very strict requirements for XML structure, particularly around namespaces:

1. **The `<bib>` root element must NOT have a default namespace declaration**
   - ❌ Rejected: `<bib xmlns="http://alma.exlibrisgroup.com/dc/01GCL_INST">`
   - ✅ Accepted: `<bib xmlns:dc="..." xmlns:dcterms="...">`
2. **Alma-specific elements (`<record>`, `<dginfo>`, etc.) must be unprefixed**
   - ❌ Rejected: `<ns0:record>` or `<alma:record>`
   - ✅ Accepted: `<record>`
3. **Dublin Core elements must use proper namespace prefixes**
   - ✅ Required: `<dc:identifier>`, `<dc:title>`, `<dcterms:created>`

The Solution

The application uses a three-step approach to handle namespaces correctly:

Step 1: Register Only Non-Default Namespaces

```
namespaces_to_register = {
    'dc': 'http://purl.org/dc/elements/1.1/',
    'dcterms': 'http://purl.org/dc/terms/',
    'xsi': 'http://www.w3.org/2001/XMLSchema-instance'
}
for prefix, uri in namespaces_to_register.items():
    ET.register_namespace(prefix, uri)
```

Note: We do NOT register the Alma default namespace (`http://alma.exlibrisgroup.com/dc/01GCL_INST`) because doing so would add an unwanted `xmlns` attribute to the `<bib>` element.

Step 2: Parse and Modify the XML

```
root = ET.fromstring(response.text)
# Find and modify elements using namespace-aware XPath
relations = root.findall('.//dc:relation', {'dc':
'http://purl.org/dc/elements/1.1/'})
```

Step 3: Clean Up Generated XML

```
xml_bytes = ET.tostring(root, encoding='utf-8')
xml_str = xml_bytes.decode('utf-8')

# Remove auto-generated ns0: prefixes from Alma elements
xml_str = xml_str.replace('ns0:', '').replace(':ns0', '')

# Remove the xmlns declaration that Alma rejects
xml_str = xml_str.replace('
xmlns="http://alma.exlibrisgroup.com/dc/01GCL_INST"', '')

xml_bytes = xml_str.encode('utf-8')
```

Why This Approach Works

When Python's `ElementTree` encounters elements in a namespace that isn't registered, it automatically generates a namespace prefix (like `ns0:`). Our post-processing step:

1. **Removes the `ns0:` prefix** - Converting `<ns0:record>` to `<record>`
2. **Removes the generated namespace declaration** - Eliminating `xmlns="http://alma.exlibrisgroup.com/dc/01GCL_INST"` from the `<bib>` element
3. **Preserves registered namespace prefixes** - Keeping `dc:`, `dcterms:`, and `xsi:` intact

Example XML Structure

What Alma Expects:

```
<bib xmlns:dc="http://purl.org/dc/elements/1.1/"
xmlns:dcterms="http://purl.org/dc/terms/">
  <mms_id>991011687640104641</mms_id>
  <title>Example Title</title>
  <record>
    <dginfo>Example dginfo</dginfo>
    <dc:identifier>dg_123456</dc:identifier>
    <dc:title>Example Title</dc:title>
  </record>
</bib>
```

What Would Be Rejected:

```
<!-- ❌ Default namespace on <bib> -->
<bib xmlns="http://alma.exlibrisgroup.com/dc/01GCL_INST">
  ...
</bib>

<!-- ❌ Namespace prefix on Alma elements -->
<bib>
  <ns0:record xmlns:ns0="http://alma.exlibrisgroup.com/dc/01GCL_INST">
```

```
    <ns0:dginfo>Example</ns0:dginfo>
  </ns0:record>
</bib>
```

Key Takeaways for Developers

1. **Always use `validate=true`** in PUT requests - Alma will tell you immediately if the XML structure is wrong
2. **Log the XML being sent** - Essential for debugging namespace issues
3. **Don't try to register the default Alma namespace** - It causes more problems than it solves
4. **Post-process the XML string** - Simple string replacement is more reliable than complex namespace handling
5. **Test with Function 1 first** - Fetch and view the XML to understand the expected structure

Related Files

- `app.py` - Contains the implementation (see `clear_dc_relation_collections()` method)
- `ALMA_API_UPDATE_NOTES.md` - Historical documentation of what approaches didn't work

Troubleshooting

"API Key not configured" error

- Ensure you've created a `.env` file with valid `ALMA_API_KEY`
- Check that the `.env` file is in the project root directory

"API Key not authorized for Sets API" error

Error message: `UNAUTHORIZED: API-key not defined or not configured to allow this API`

This error occurs when trying to load a set without proper API key permissions.

Solution:

1. Log into Alma as an administrator
2. Navigate to: **Admin > General > API Key Management**
3. Find and edit your API key
4. Under **API Permissions**, ensure these are enabled:
   - **Bibs**: Read/Write ✓
   - **Configuration**: Read ✓ (this is required for Sets API)
5. Click **Save**
6. Wait a few minutes for the changes to propagate
7. Restart the application and try again

Note: If you don't have admin access, contact your Alma administrator to add Configuration permissions to your API key.

"Failed to connect to Alma API" error

- Verify your API key is valid
- Check that `ALMA_API_REGION` is set to one of: `America`, `Europe`, `Asia Pacific`, `Canada`, or `China`
- Ensure your API key has appropriate permissions
- Confirm the region matches your institution's Alma location

Namespace-related errors (400 BAD_REQUEST)

Error: `unexpected element (uri:"...", local:"bib"). Expected elements are <{}bib>`

- This means the `<bib>` element has a namespace declaration it shouldn't have
- Solution is already implemented in the code's string replacement step

Error: `Field ns0:dginfo is invalid`

- This means Alma elements have an unwanted namespace prefix
- Solution is already implemented in the code's `ns0:` removal step

Error: `Field dc:identifier is invalid`

- Check that Dublin Core namespaces are properly registered
- Verify the namespace URIs are exactly correct

Virtual environment issues

- Delete the `.venv` folder and run `./run.sh` again
- Ensure Python 3.8+ is installed: `python3 --version`

Contributing

Contributions are welcome! Please:

1. Fork the repository
2. Create a feature branch
3. Make your changes
4. Submit a pull request

License

[Specify license here]

Contact

For questions or support, please open an issue on GitHub.

Acknowledgments

- Built with Flet
- Developed for Digital Grinnell
- Alma API integration using direct REST calls